Clustering with QAOA

Community Detection via Hierarchical Modularity Maximization using the Quantum Approximate Optimization Algorithm

Sindre Kampen Nesheim

FYS5419 — Quantum Computing and Quantum Machine Learning
2026

Abstract

We present H-QAOA, a hierarchical bisection algorithm for graph community detection based on the Quantum Approximate Optimization Algorithm (QAOA). Starting from a modularity maximization objective, we derive a diagonal Ising Hamiltonian and implement a full statevector simulator from scratch using `NumPy` and `SciPy`. The $k = 2$ binary Ising encoding restricts each QAOA instance to exactly V qubits, and multi-community structure is recovered by recursively bisecting subgraphs. We benchmark H-QAOA on four graph instances of up to 12 nodes against a brute-force ground truth, finding that H-QAOA with circuit depth $p = 1$ recovers the exact optimal partition in all four cases, with modularity values between $Q = 0.22$ and $Q = 0.39$. We further characterize the variational landscape, demonstrating that the expected value $\langle H_C \rangle$ is π-periodic in the mixer parameter β due to the R_X gate structure, while the landscape in the phase separation parameter γ is complex and graph-dependent due to the eigenvalue spectrum of H_C. Increasing the circuit depth p from 1 to 6 improves the optimal configuration probability from $P = 0.043$ to $P = 0.375$ on the `SmallKarateClub` instance, consistent with the theoretical guarantee that the QAOA ansatz converges to the exact ground state as $p \to \infty$. The source code is publicly available at https://github.com/sindrekn/HierarchicalQAOAClustering.

Contents

1 Introduction

Graph clustering, or community detection, is a fundamental problem in network science: given a graph $G = (V, E)$, partition its nodes into groups such that edges within groups are dense and edges between groups are sparse. Applications span a wide range of domains, from identifying functional modules in biological networks and detecting communities in social graphs [1]. Graph clustering is NP-hard, and heuristic solvers such as the Louvain algorithm [2] is needed to solve them.

The emergence of quantum computing offers a new method for combinatorial optimization. The Quantum Approximate Optimization Algorithm (QAOA), introduced by Farhi, Goldstone, and Gutmann [3], is a hybrid classical-quantum algorithm that encodes an optimization problem into a parameterized quantum circuit and uses a classical optimizer to tune the circuit parameters toward the optimal solution. Using QAOA to solve community detection via modularity maximization is a natural target: the modularity objective maps cleanly onto a two-body Ising Hamiltonian, and the resulting cost landscape can be evaluated efficiently on a quantum processor.

A key practical limitation of direct QAOA approaches to community detection is the encoding cost. A general k-community partition requires at least $k \cdot V$ binary variables when using super-node encodings [4], making the qubit count grow rapidly with both the number of nodes and the number of communities. In this work we exploit the binary nature of the Ising spin variables to restrict each QAOA instance to exactly V qubits, sufficient for a two-community split, and recover multi-community structure through a *hierarchical bisection* strategy. The graph is recursively partitioned by applying the binary QAOA to progressively smaller subgraphs, accepting each split only if it improves the global modularity of the original graph. This approach, which we refer to as H-QAOA, builds a binary bisection tree whose leaves define the final community assignment.

The hierarchical bisection idea connects to a classical tradition of divisive graph partitioning [5] and to recent quantum annealing work on recursive modularity maximization [6], but to my knowledge has not previously been combined with a statevector QAOA simulation in a fully from-scratch Python implementation. The from-scratch approach, using only `NumPy` and `SciPy`, with no dependence on `Qiskit` or other quantum frameworks, ensures complete transparency over the simulation and provides a clean foundation for future benchmarking against classical solvers such as Simulated Annealing, Simulated Quantum Annealing, and Simulated Bifurcation.

The remainder of this paper is organized as follows. Section 2 derives the modularity Hamiltonian and the QAOA circuit. Section 3 describes the implementation and introduces the H-QAOA bisection algorithm. Section 4 presents results on four graph instances and characterizes the variational parameter landscape. Section 5 summarizes the findings and outlines directions for further research.

2 Methods

2.1 Modularity and the Cost Hamiltonian

2.1.1 Modularity

Community detection seeks to partition the nodes of a graph $G = (V, E)$, where V is the set of vertices or nodes and E the set of edges, into communities such that intra-community edges are dense and inter-community edges are sparse. We quantify the quality of a partition using the *modularity* Q [7]:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \alpha \frac{k_i k_j}{2m} \right] \delta(c_i, c_j), \tag{1}$$

where A_{ij} is the (symmetric) adjacency matrix, $k_i = \sum_j A_{ij}$ is the degree of node i, $m = \frac{1}{2} \sum_{ij} A_{ij}$ is the total number of edges, c_i is the community label of node i, α serves as a resolution parameters, and $\delta(c_i, c_j)$ is the Kronecker delta, which is unity when nodes i and j belong to the same community and zero otherwise. The null model term $k_i k_j / 2m$ represents the expected edge density under a random graph with the same degree sequence, so Q measures the deviation of the actual edge structure from this random baseline. A partition is generally considered to have significant community structure when $Q \gtrsim 0.3$ [7].

2.1.2 Ising Encoding

To cast modularity maximization as an optimization problem for quantum algorithms, we map the community labels onto Ising spin variables $s_i \in \{+1, -1\}$, where the two spin values encode membership in one of two communities. The Kronecker delta can then be written exactly as:

$$\delta(c_i, c_j) = \frac{s_i s_j + 1}{2}, \tag{2}$$

since $s_i s_j = +1$ when both spins share the same value (same community) and $s_i s_j = -1$ otherwise, giving 1 and 0 respectively. Substituting into equation (1) yields:

$$Q = \frac{1}{2m} \sum_{ij} \left[A_{ij} - \alpha \frac{k_i k_j}{2m} \right] \frac{s_i s_j + 1}{2}. \tag{3}$$

Introducing the *modularity matrix* B_{ij} [8]:

$$B_{ij} = \frac{1}{2m} \left[A_{ij} - \alpha \frac{k_i k_j}{2m} \right], \tag{4}$$

equation (3) separates into an interaction term and a constant offset:

$$Q = \frac{1}{2} \sum_{ij} B_{ij} \, s_i s_j + \frac{1}{2} \sum_{ij} B_{ij}. \tag{5}$$

This binary encoding uses exactly $N = V$ spin variables (one per node) in contrast to a general k-community encoding via super-nodes, which requires at least $N \geq V \cdot k$ binary variables [4], and most likely more due to penalty constrains. The restriction to $k = 2$ communities is overcome by applying the encoding recursively, as described in Section 3.2.

2.1.3 Cost Hamiltonian

To embed the optimization problem in a quantum circuit we promote the classical spin variables to Pauli-Z operators, $s_i \to Z_i$, acting on qubit i in the N-qubit Hilbert space. A single-qubit Z operator on qubit i is constructed via the Kronecker product:

$$Z_i = I^{\otimes i} \otimes \sigma_z \otimes I^{\otimes(N-i-1)}, \tag{6}$$

where qubit 0 is the leftmost (most significant) factor. Two-qubit interactions $s_i s_j \to Z_i Z_j$ are embedded analogously:

$$Z_i Z_j = I^{\otimes i} \otimes \sigma_z \otimes I^{\otimes(j-i-1)} \otimes \sigma_z \otimes I^{\otimes(N-j-1)}. \tag{7}$$

The full $(2^N \times 2^N)$ cost Hamiltonian is then:

$$H_C = \sum_{ij} B_{ij}\, Z_i Z_j \;+\; C \cdot I^{\otimes N}, \tag{8}$$

where $C = \frac{1}{2}\sum_{ij} B_{ij}$ is the constant offset inherited from equation (5). Since H_C is diagonal in the computational basis, its 2^N eigenvalues are precisely the modularity values of all possible two-community partitions, and maximizing Q is equivalent to finding the ground state of $-H_C$.

2.2 Quantum Approximate Optimization Algorithm

2.2.1 Mixer Hamiltonian

Before introducing the formal QAOA circuit we define the mixer (or driver) Hamiltonian H_M:

$$H_M = \sum_{i=0}^{N-1} X_i, \qquad X_i = I^{\otimes i} \otimes \sigma_x \otimes I^{\otimes(N-i-1)}, \tag{9}$$

where σ_x is the Pauli-X matrix. The ground state of H_M is the uniform superposition $|+\rangle^{\otimes N}$, with $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, which serves as the initial state of the QAOA circuit. Crucially, H_M and H_C do not commute,

$$[H_C,\, H_M] \neq 0, \tag{10}$$

because σ_z and σ_x do not commute on the same qubit. This non-commutativity is essential: it means the two unitaries generate independent directions in Hilbert space, allowing the circuit to coherently explore the energy landscape in a way that cannot be replicated by either unitary alone.

2.2.2 Circuit Definition

The depth-p QAOA ansatz prepares the variational state [3]:

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = \prod_{l=1}^{p} e^{-i\beta_l H_M} e^{-i\gamma_l H_C} |+\rangle^{\otimes N}, \tag{11}$$

where $\boldsymbol{\gamma} = (\gamma_1, \ldots, \gamma_p)$ and $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_p)$ are variational parameters, and p is the circuit depth controlling the length of the ansatz. Each layer consists of two alternating unitaries: the problem unitary $e^{-i\gamma_l H_C}$, which encodes the cost function, and the mixing unitary $e^{-i\beta_l H_M}$, which drives transitions between computational basis states and prevents the optimizer from becoming trapped in a single configuration. In the limit $p \to \infty$ the ansatz can represent the exact ground state of H_C [3], and in practice increasing p monotonically improves the approximation ratio.

The variational objective is to find the parameters that maximize the expectation value of the cost Hamiltonian:

$$F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta}| H_C |\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle. \tag{12}$$

Since H_C is diagonal in the computational basis, equation (12) reduces to a weighted sum over basis state probabilities:

$$F_p(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \sum_{\mathbf{x}\in\{0,1\}^N} |\langle \mathbf{x}|\psi\rangle|^2 \, E(\mathbf{x}), \tag{13}$$

where $E(\mathbf{x}) = \langle \mathbf{x}|H_C|\mathbf{x}\rangle$ are the diagonal entries of H_C and $|\langle \mathbf{x}|\psi\rangle|^2$ are the measurement probabilities. This avoids explicit matrix–vector products with the full $2^N \times 2^N$ Hamiltonian, reducing the cost of each evaluation. In the result part of the paper $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ will be denoted as $\langle H_C \rangle$.

2.2.3 Hybrid Classical–Quantum Optimization Loop

QAOA is a hybrid algorithm: the quantum circuit prepares the state and evaluates F_p, while a classical optimizer updates the parameters. The procedure is as follows:

1. Initialize $(\boldsymbol{\gamma}, \boldsymbol{\beta})$ randomly or by choice.

2. Prepare the state $|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle$ by running the quantum circuit of equation (11).

3. Evaluate $F_p(\boldsymbol{\gamma}, \boldsymbol{\beta})$ via equation (13).

4. Pass $-F_p$ to a classical optimizer (we use COBYLA [9] by `SciPy`), which proposes updated parameters $(\boldsymbol{\gamma}, \boldsymbol{\beta})$.

5. Repeat steps 2–4 until the optimizer converges.

6. Measure the final state in the computational basis; the most probable bitstring gives the candidate partition.

To mitigate convergence to local optima, the procedure is repeated from multiple random initializations and the globally best result is retained, this is named `n_restarts` and the best result from all the restarts is chosen.

2.2.4 Unitary Implementation

Cost unitary. The phase separation unitary $U_C(\gamma) = e^{-i\gamma H_C}$ is trivial to apply since H_C is diagonal: it reduces to elementwise multiplication of the statevector by the phase factors:

$$U_C(\gamma)\,|\psi\rangle = e^{-i\gamma E(\mathbf{x})}|\psi\rangle, \tag{14}$$

where the exponential acts independently on each amplitude.

Mixer unitary. The mixing unitary $U_M(\beta) = e^{-i\beta H_M}$ requires more care since H_M is not diagonal. However, because all terms X_i in equation (9) act on different qubits they commute with each other, $[X_i, X_j] = 0$ for $i \neq j$, and the exponential therefore factorises exactly into a tensor product of single-qubit rotations:

$$e^{-i\beta H_M} = \bigotimes_{i=0}^{N-1} e^{-i\beta X_i} = \bigotimes_{i=0}^{N-1} R_X(2\beta), \tag{15}$$

where $R_X(\theta) = e^{-i\frac{\theta}{2}\sigma_x}$ is the single-qubit rotation gate. This means $U_M(\beta)$ can be applied qubit by qubit without constructing the full $2^N \times 2^N$ matrix.

3 Code and Benchmarking

3.1 Code Structure

The implementation is written in Python 3.12.3 using `NumPy` 2.4.4 and `SciPy` 1.17.1 for the statevector simulation and classical optimization respectively. The implementation does not rely on `Qiskit` or other quantum computing frameworks; this is a deliberate choice to maintain full transparency over the simulation and to build understanding

of the algorithm from first principles. The complete source code is available at https://github.com/sindrekn/HierarchicalQAOAClustering. The principal components of the implementation are:

`class QAOAClustering` Performs the statevector simulation of the QAOA circuit. Stores the diagonal of the cost Hamiltonian H_C and applies the cost and mixer unitaries as described in Section 2.2.

`def qaoa_k2_cluster` Wraps `QAOAClustering` with a classical COBYLA optimizer and performs multiple random restarts, returning the optimal variational parameters and the most probable bitstring partition.

`def HierarchicalBisection` Implements the recursive H-QAOA bisection algorithm described in Section 3.2. Manages the bisection tree, global modularity tracking, and the α scaling schedule.

`def pauli_z_hamiltonian` Constructs the full $(2^N \times 2^N)$ cost Hamiltonian H_C in the computational basis via successive Kronecker products, as described in Section 2.1.3.

3.2 Hierarchical QAOA Bisection (H-QAOA)

As established in Section 2, the binary Ising encoding restricts the QAOA optimization to a two-community partition ($k = 2$). To recover multi-community structure without introducing super-nodes, we employ a *divisive hierarchical bisection* scheme [10], which we refer to as H-QAOA. The algorithm builds a binary bisection tree top-down: at each node of the tree, QAOA partitions the current subgraph into two candidate sub-communities, and the split is accepted or rejected based on whether it improves the *global* modularity of the original graph G.

The procedure is as follows:

1. **Initialization.** The full graph $G = (V, E)$ is treated as a single community with label 0. The current best global modularity $Q^* = Q(\mathbf{c}_0)$ is computed for the trivial single-community assignment $\mathbf{c}_0$ (this is always 0).

2. **QAOA Bisection.** For the current subgraph $G_s = (V_s, E_s)$, a cost Hamiltonian $H_C^{(s)}$ is constructed and QAOA is executed. The most probable output bitstring $\mathbf{x}^* \in \{0, 1\}^{V_s}$ partitions V_s into two candidate sub-communities V_0 and V_1.

3. **Global Modularity Evaluation.** The proposed partition is mapped back to the original node indexing of G, and the global modularity Q_{prop} is evaluated on the full graph G under the updated community assignment.

4. **Accept/Reject.** If $Q_{\text{prop}} > Q^*$, the split is accepted: Q^* is updated, the new cluster label is committed, and both child subgraphs G_{V_0} and G_{V_1} are recursively bisected from step 2. If $Q_{\text{prop}} \leq Q^*$, the split is rejected and V_s is finalized as a leaf community.

The recursion terminates when every active subgraph either fails the accept criterion, falls below a minimum size `min_cluster_size`, or reaches the maximum bisection level `max_level`. A simplified pseudocode representation of the algorithm is given in Listing 1.

```
def HierarchicalBisection(A: np.ndarray):
    best_modularity = modularity_calc(A, x=zeros(N))

    def _bisect(subA, global_indices, current_labels):
        local_config = qaoa_k2_cluster(subA)
        mask0 = (local_config == 0)
        mask1 = (local_config == 1)
        if mask0.sum() == 0 or mask1.sum() == 0:
            return  # trivial split     reject
        proposed_labels = current_labels.copy()
        proposed_labels[global_indices[mask1]] = new_cluster_id
        proposed_modularity = modularity_calc(A, x=proposed_labels)
        if proposed_modularity > best_modularity:
            # Accept: recurse into both child subgraphs
            _bisect(subA[ix_(mask0, mask0)], global_indices[mask0],
                ...)
            _bisect(subA[ix_(mask1, mask1)], global_indices[mask1],
                ...)
        # else: reject, finalize this subgraph as a leaf community

    _bisect(subA=A, global_indices=arange(N), current_labels=zeros(
       N))
```

Listing 1: Simplified pseudocode for the H-QAOA hierarchical bisection. The full implementation is available on GitHub.

Resolution parameter scaling. The modularity resolution parameter α (equation 1) is scaled at each bisection level by a factor $\alpha_{\text{scale}} > 1$:

$$\alpha^{(\ell)} = \alpha^{(0)} \cdot \alpha_{\text{scale}}^{\ell}, \tag{16}$$

where ℓ is the current bisection level. As the subgraphs become smaller and more densely connected, the null model term $k_i k_j / 2m$ increasingly dominates and suppresses

the modularity gain from any further split. Scaling α upward weakens this null model penalty, making it easier for QAOA to identify community structure within tight subgraphs. Importantly, the accept/reject criterion always evaluates the *global* modularity on the original graph with $\alpha = 1$, so the scaling of α only affects the QAOA search, not the acceptance decision.

3.3 Benchmarks

To evaluate H-QAOA we constructed four benchmark graphs of varying size and community structure: `XXSgraph`, `HomeMadeGraph1`, `HomeMadeGraph2`, and `SmallKarateClub`. Graph sizes are limited to at most 12 vertices due to the exponential scaling of the statevector simulation. `XXSgraph`, `HomeMadeGraph1`, and `HomeMadeGraph2` are hand-crafted graphs designed with known community structures of varying density. `SmallKarateClub` is the induced subgraph of the first 12 nodes of Zachary's karate club graph [11], a standard community detection benchmark with 34 nodes in its full form.

H-QAOA results are compared against a brute-force ground truth obtained by exhaustively evaluating the modularity of all k^N community assignments for $k = 2, \ldots, k_{\text{opt}} + 1$, where k_{opt} is the number of communities returned by H-QAOA. The upper limit $k_{\text{opt}} + 1$ is included to verify that the algorithm has not terminated prematurely, since a higher k could in principle yield a higher modularity even when the k_{opt} result is locally optimal.

The H-QAOA parameters used to identify the community structure are: circuit depth $p = 1$, `n_restarts` $= 40$, $\gamma \in [0, 4\pi]$, $\beta \in [0, \pi]$, $\alpha^{(0)} = 1.0$, and $\alpha_{\text{scale}} = 1.5$.

Graph	**H-QAOA** Q	**Brute Force** Q	V	E	k_{opt}
`XXSgraph`	0.2200	0.2200	5	5	2
`HomeMadeGraph1`	0.3933	0.3933	10	15	3
`HomeMadeGraph2`	0.3571	0.3571	12	14	3
`SmallKarateClub`	0.2717	0.2717	12	22	2

Table 1: Modularity Q returned by H-QAOA versus the brute-force ground truth, together with graph size (V vertices, E edges) and the optimal community count k_{opt} found by H-QAOA. H-QAOA recovers the exact optimum in all four cases.

Table 1 shows that H-QAOA recovers the exact brute-force optimum on all four benchmark instances. The resulting graph partitions are visualized in Appendix A. These results demonstrate that even at circuit depth $p = 1$, the hierarchical bisection strategy is sufficient to identify the globally optimal community structure for the graph sizes considered here. Whether this holds for larger graphs where the QAOA landscape becomes harder to optimize is an open question that motivates future work at higher p.

4 Results and Discussion

4.1 Graph Partition

Table 1 shows that H-QAOA recovers the exact brute-force optimal modularity on all four benchmark instances. While the graph sizes are small enough that perfect recovery is achievable in principle, it is not guaranteed, the QAOA optimizer could converge to suboptimal local minima, and the hierarchical bisection could accept an early split that rules out the global optimum. The perfect agreement with the brute-force ground truth therefore validates both the Ising Hamiltonian formulation and the hierarchical bisection strategy.

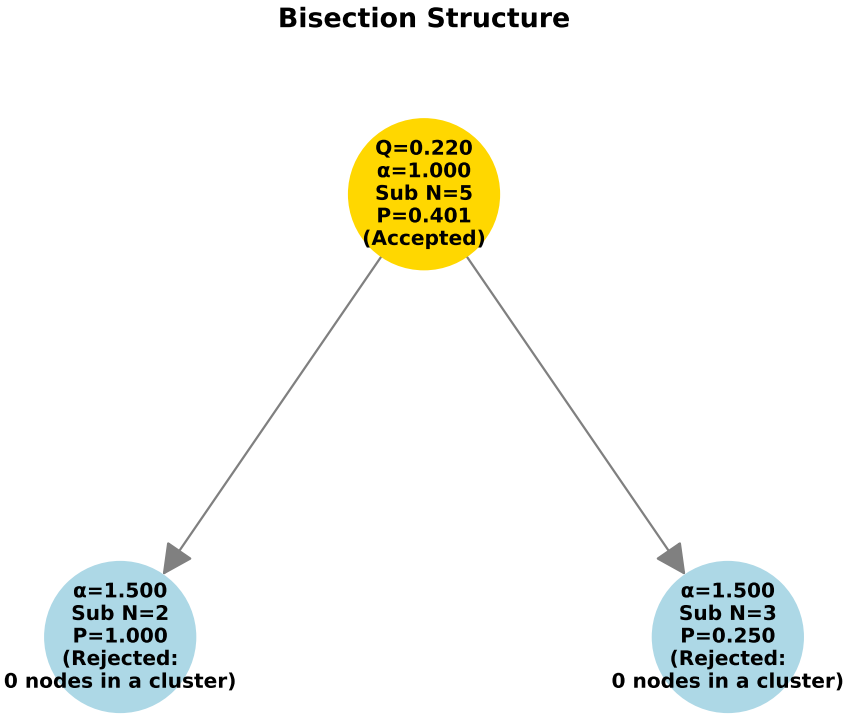

Figure 1: Bisection tree for the `XXSgraph` instance at circuit depth $p = 1$. Each node reports the global modularity Q, resolution parameter α, subgraph size Sub N, and optimal configuration probability P.

To gain insight into how the algorithm reaches these solutions, figures 1 and 2 show the bisection trees for the `XXSgraph` and `HomeMadeGraph1` instances at depth $p = 1$. Each node in the tree reports the global modularity Q, the resolution parameter α, the subgraph node count Sub N, and the probability P of the optimal bitstring, defined as the combined probability of the two degenerate $\mathbb{Z}_2$-symmetric configurations, since relabeling cluster 0 as cluster 1 and vice versa produces an identical partition with equal modularity.

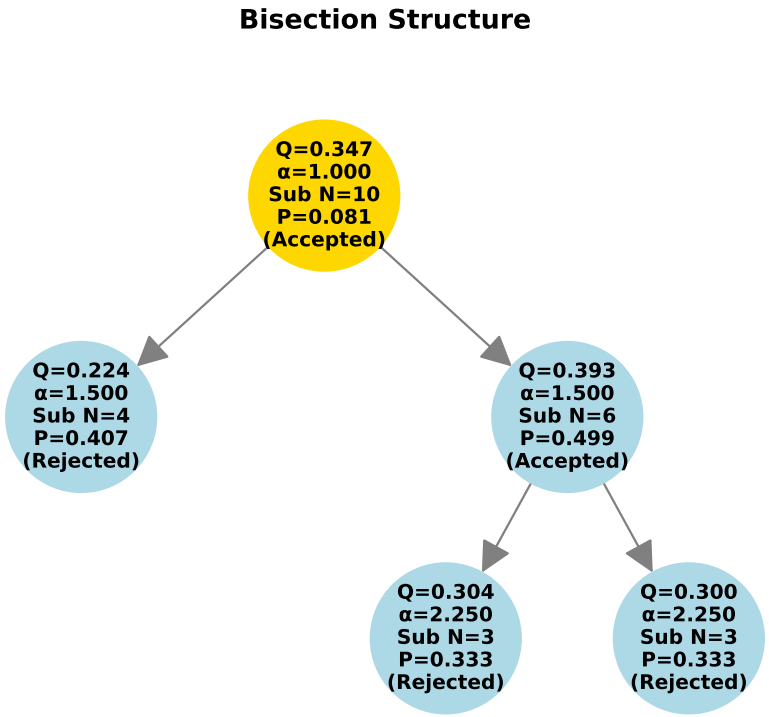

Figure 2: Bisection tree for the `HomeMadeGraph1` instance at circuit depth $p = 1$.

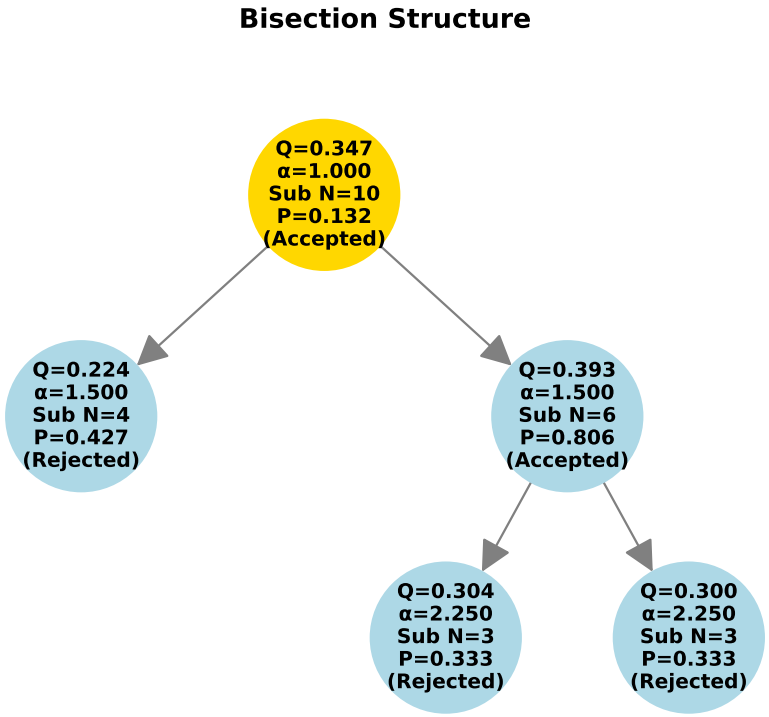

Figure 3: Bisection tree for the `HomeMadeGraph1` instance at circuit depth $p = 2$.

A clear trend is visible across both trees: P increases as Sub N decreases. This

is expected, smaller subgraphs have lower-dimensional Hilbert spaces and simpler cost landscapes, making it easier for the $p = 1$ ansatz to concentrate probability on the optimal bitstring.

To investigate the effect of circuit depth on the bisection quality, we repeat the H-QAOA run for `HomeMadeGraph1` at $p = 2$, with all other hyperparameters held fixed. The resulting bisection tree is shown in figure 3.

Comparing figures 2 and 3, the accepted bisections show a substantial increase in P when moving from $p = 1$ to $p = 2$, while rejected bisections remain largely unchanged. This asymmetry is physically meaningful: accepted bisections correspond to subgraphs with a well-defined community structure, producing a cost landscape with a sharp global maximum that the deeper $p = 2$ ansatz can more reliably locate. Rejected bisections, by contrast, correspond to subgraphs where the modularity differences between configurations are small, a nearly flat energy landscape that offers little signal to the optimizer regardless of circuit depth. This suggests that the accept/reject threshold serves not only as a modularity criterion but also as an implicit filter on problem hardness.

4.2 QAOA Parameter Landscape

4.2.1 Depth Dependence

To isolate the effect of circuit depth from the hierarchical bisection strategy, we run the 2-cluster QAOA directly on the `SmallKarateClub` instance for depths $p \in \{1, 2, 3, 4, 5, 6\}$, using `n_restarts` $= 40$, $\gamma \in [0, 4\pi]$, $\beta \in [0, \pi]$, and $\alpha = 1$. Figure 4 shows the probability distribution over the ten most probable bitstrings at each depth, together with the combined $\mathbb{Z}_2$ probability P and the optimal expected value $\langle H_C \rangle$.

Both P and $\langle H_C \rangle$ increase monotonically with p, consistent with the theoretical guarantee that the QAOA ansatz can represent the exact ground state in the limit $p \to \infty$ [3]. The two highest-probability bitstrings at every depth are bitwise complements of each other, confirming the $\mathbb{Z}_2$ degeneracy of the optimal partition.

4.2.2 (γ, β) Landscape

To characterize the variational landscape, we perform a dense grid scan over (γ, β) using 100×100 evenly spaced points, evaluating $\langle H_C \rangle$ at each point without any classical optimization. γ and β ranges over $[0, 8\pi] \times [0, 2\pi]$ for the small `XXSgraph`, while for the three other graphs: $(\gamma, \beta) \in [0, 250\pi] \times [0, 2\pi]$. Heatmaps for `XXSgraph` and `SmallKarateClub` are shown in figures 5 and 6; results for `HomeMadeGraph1` and `HomeMadeGraph2` are in Appendix B.

Both figures exhibit a clear period of π in β and a complex, graph-dependent structure in γ, consistent with results reported by Stęchły et al. [12]. The origins of these two

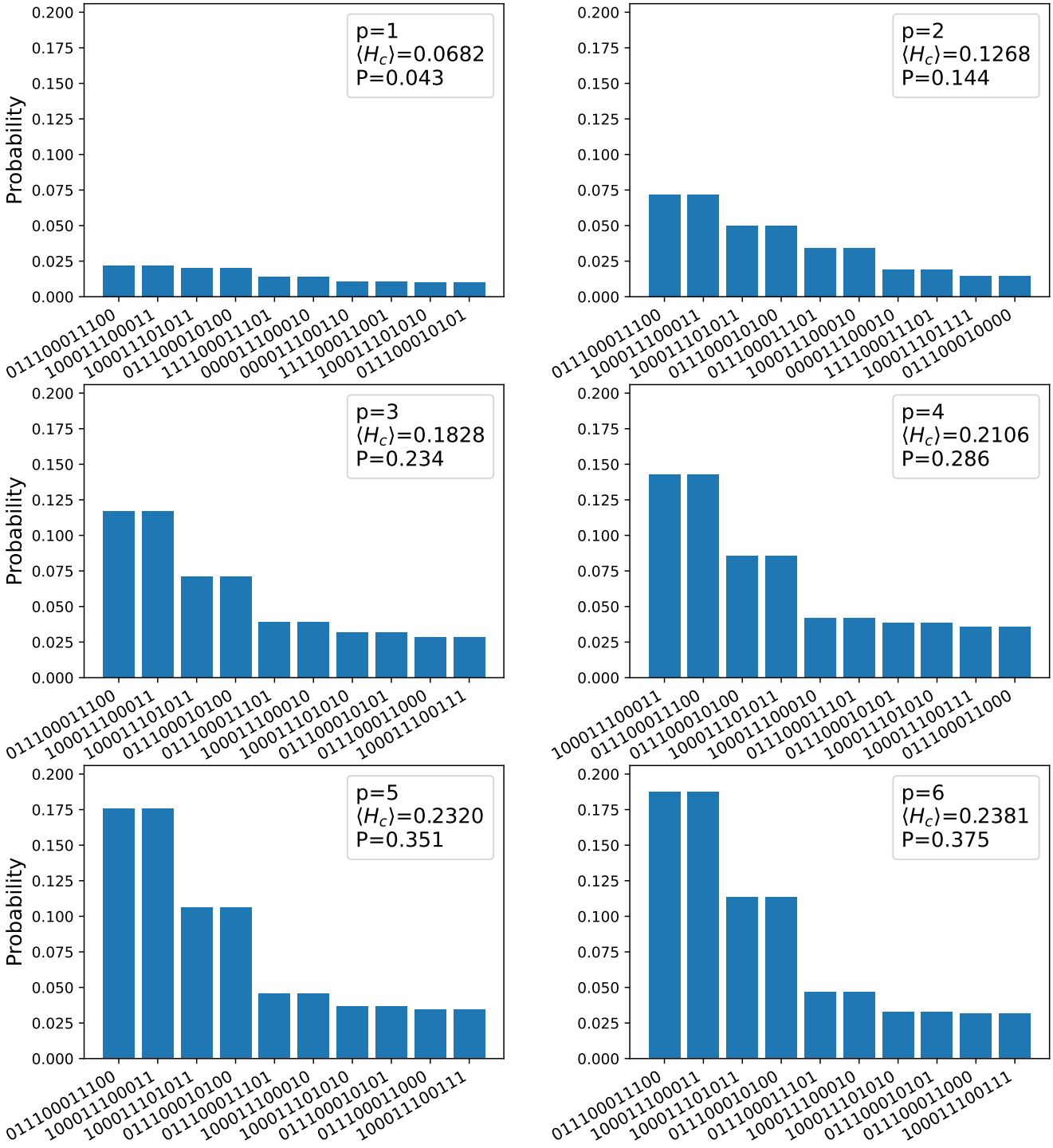

Figure 4: Probability distributions over the top 10 bitstrings for the `SmallKarateClub` instance at depths $p \in \{1, \ldots, 6\}$.

behaviors are distinct and worth examining separately.

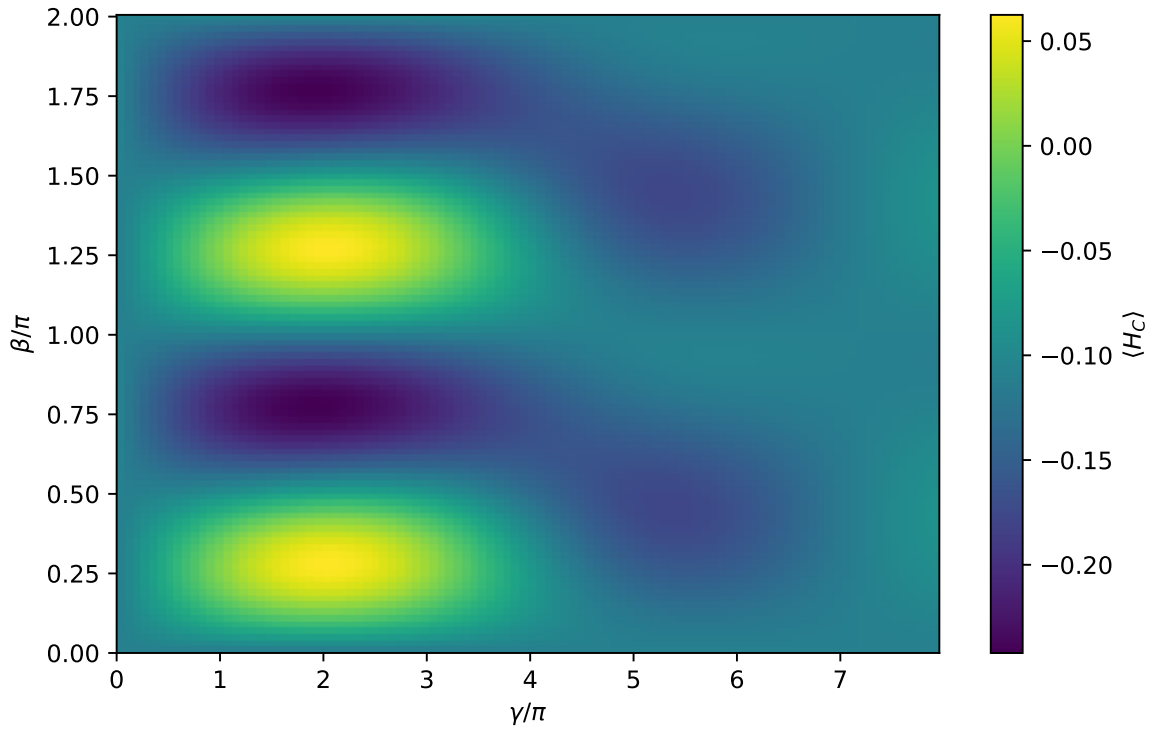

Figure 5: $\langle H_C \rangle$ over the $(\gamma/\pi,\, \beta/\pi)$ grid for the `XXSgraph` instance.

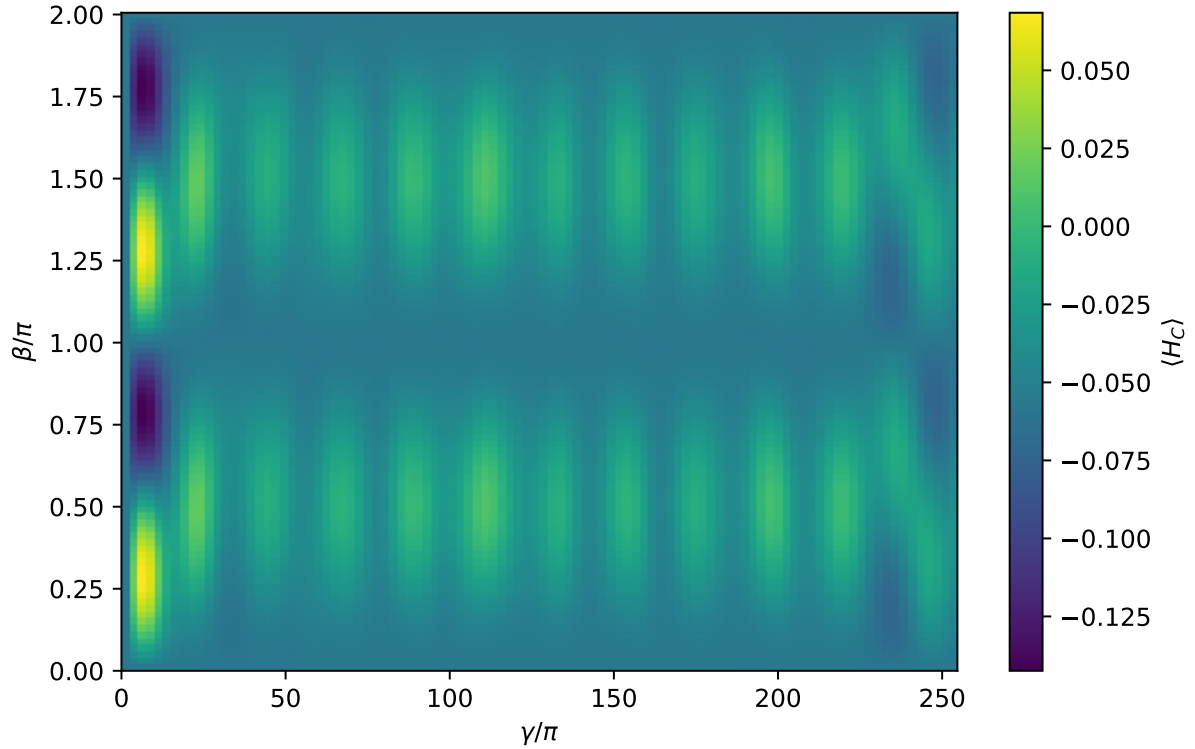

Figure 6: $\langle H_C \rangle$ over the $(\gamma/\pi,\, \beta/\pi)$ grid for the `SmallKarateClub` instance.

Periodicity in β. The mixer unitary factorises into single-qubit R_X rotations (equation 15):

$$e^{-i\beta H_M} = \bigotimes_i R_X(2\beta), \qquad R_X(\theta) = \cos\tfrac{\theta}{2}\, I - i \sin\tfrac{\theta}{2}\, \sigma_x. \tag{17}$$

Since $R_X(2\beta)$ is 2π-periodic in 2β, it is π-periodic in β. The landscape is therefore exactly π-periodic in β for any graph, independently of the cost Hamiltonian. This is why the optimal β can always be found within $[0, \pi/2]$ [3]. It is worth to note that if you change the mixer Hamiltonian, this does no longer hold.

Complex structure in γ. The γ parameter controls the time evolution of the cost Hamiltonian (H_C). Because H_C acts on the computational basis states with different eigenvalues, the unitary operator $e^{-i\gamma H_C}$ applies a distinct phase rotation to each state component in the quantum superposition. When these phase states interfere during the subsequent mixer steps, they create a complex quantum interference pattern. For general optimization problems with non-integer or varied weights, these phases rotate at mismatched frequencies that do not cleanly align, causing the expected energy landscape to exhibit a highly non-linear, rugged, and non-periodic structure full of local minima. For a further explanation see the paper by Stęchły et al. [12].

5 Conclusions and Further Research

Community detection in graphs is a combinatorially hard problem with applications across network science, biology, and social systems. In this work we have developed H-QAOA, a hierarchical bisection algorithm that solves multi-community graph clustering by recursively applying a from-scratch QAOA statevector simulation to the binary modularity maximization problem.

The results establish three main findings. First, H-QAOA with $p = 1$ recovers the brute-force optimal partition on all four benchmark instances, validating the correctness of the Ising formulation, the $\mathbb{Z}_2$-symmetric binary encoding and bisection strategy. Second, increasing the circuit depth p improves both the success probability P and the expected value $\langle H_C \rangle$, with the largest gains on subgraphs with well-defined community structure, while nearly flat energy landscapes remain hard regardless of depth, suggesting that the accept/reject criterion implicitly filters problem hardness. Third, the (γ, β) landscape has a universal π-periodicity in β arising from the R_X gate structure, while the γ landscape is complex and graph-dependent due to the eigenvalue spectrum of H_C, motivating wide search ranges and multiple random restarts in the optimizer

The main limitation of the current approach is exponential growth of the statevector with the number of qubits, restricting exact simulation to $n \lesssim 13$ on a laptop. Future work could include:

- Extending to real quantum hardware such as the IBM quantum computers using the Qiskit Runtime API. This could give us the possibility to test the algorithm on bigger graphs and study the performance under hardware noise.

- A deeper study of the γ dependency in $\langle H_C \rangle$, building on the work by Stęchły et al. [12].
- Using the Hierarchical bisection function with other QUBO solvers, such as Simulated Annealing, Simulated Quantum Annealing, Simulated Bifurcation. This could provide a benchmark comparison across different solvers.

References

[1] Santo Fortunato. "Community detection in graphs". In: *Physics Reports* 486.3-5 (Feb. 2010), pp. 75–174. ISSN: 0370-1573. DOI: 10.1016/j.physrep.2009.11.002. URL: http://dx.doi.org/10.1016/j.physrep.2009.11.002.

[2] Vincent D Blondel et al. "Fast unfolding of communities in large networks". en. In: *Journal of Statistical Mechanics: Theory and Experiment* 2008.10 (Oct. 2008), P10008. ISSN: 1742-5468. DOI: 10.1088/1742-5468/2008/10/P10008. URL: https://iopscience.iop.org/article/10.1088/1742-5468/2008/10/P10008 (visited on 05/06/2026).

[3] Edward Farhi, Jeffrey Goldstone and Sam Gutmann. *A Quantum Approximate Optimization Algorithm*. en. arXiv:1411.4028 [quant-ph]. Nov. 2014. DOI: 10.48550/arXiv.1411.4028. URL: http://arxiv.org/abs/1411.4028 (visited on 07/05/2026).

[4] Christian F. A. Negre, Hayato Ushijima-Mwesigwa and Susan M. Mniszewski. "Detecting multiple communities using quantum annealing on the D-Wave system". en. In: *PLOS ONE* 15.2 (Feb. 2020). Ed. by Itay Hen, e0227538. ISSN: 1932-6203. DOI: 10.1371/journal.pone.0227538. URL: https://dx.plos.org/10.1371/journal.pone.0227538 (visited on 05/06/2026).

[5] B. W. Kernighan and S. Lin. "An efficient heuristic procedure for partitioning graphs". In: *The Bell System Technical Journal* 49.2 (1970), pp. 291–307. DOI: 10.1002/j.1538-7305.1970.tb01770.x.

[6] Joan Falcó-Roget et al. "Modularity Maximization and Community Detection in Complex Networks Through Recursive and Hierarchical Annealing in the DWAVE Advantage Quantum Processing Units". In: *IEEE Transactions on Network Science and Engineering* 13 (2026), pp. 7394–7410. ISSN: 2327-4697. DOI: 10.1109/TNSE.2026.3668505. URL: https://ieeexplore.ieee.org/document/11417306/ (visited on 05/06/2026).

[7] M. E. J. Newman and M. Girvan. "Finding and evaluating community structure in networks". en. In: *Physical Review E* 69.2 (Feb. 2004), p. 026113. ISSN: 1539-3755, 1550-2376. DOI: 10.1103/PhysRevE.69.026113. URL: https://link.aps.org/doi/10.1103/PhysRevE.69.026113 (visited on 07/05/2026).

[8] M. E. J. Newman. “Finding community structure in networks using the eigenvectors of matrices”. en. In: *Physical Review E* 74.3 (Sept. 2006), p. 036104. ISSN: 1539-3755, 1550-2376. DOI: 10.1103/PhysRevE.74.036104. URL: https://link.aps.org/doi/10.1103/PhysRevE.74.036104 (visited on 05/06/2026).

[9] Michael J. D. Powell. “A direct search optimization method that models the objective and constraint functions by linear interpolation”. In: *Advances in Optimization and Numerical Analysis*. Ed. by Susana Gomez and Jean-Pierre Hennart. Vol. 275. Mathematics and Its Applications. Springer, 1994, pp. 51–67. DOI: 10.1007/978-94-015-8330-5_4.

[10] Boris Mirkin. “Hierarchical Clustering”. en. In: *Core Concepts in Data Analysis: Summarization, Correlation and Visualization*. Series Title: Undergraduate Topics in Computer Science. London: Springer London, 2011, pp. 283–313. ISBN: 978-0-85729-286-5 978-0-85729-287-2. DOI: 10.1007/978-0-85729-287-2_7. URL: http://link.springer.com/10.1007/978-0-85729-287-2_7 (visited on 05/06/2026).

[11] Wayne W. Zachary. “An Information Flow Model for Conflict and Fission in Small Groups”. en. In: *Journal of Anthropological Research* 33.4 (1977), pp. 452–473. URL: http://www.jstor.org/stable/3629752.

[12] Michał Stęchły et al. *Connecting the Hamiltonian structure to the QAOA energy and Fourier landscape structure*. en. arXiv:2305.13594 [quant-ph]. May 2024. DOI: 10.48550/arXiv.2305.13594. URL: http://arxiv.org/abs/2305.13594 (visited on 05/06/2026).

A Graph Partitions

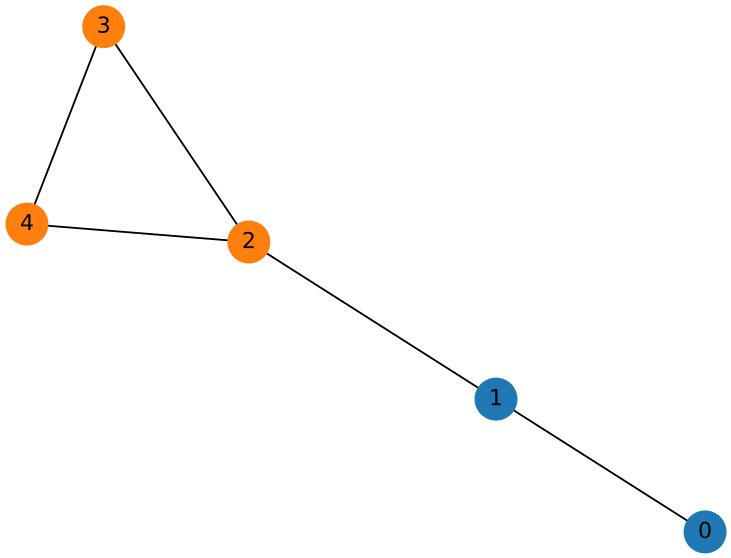

Figure 7: Optimal found solution from H-QAOA on `XXSgraph` instance. Q=0.22 with k=2 clusters.

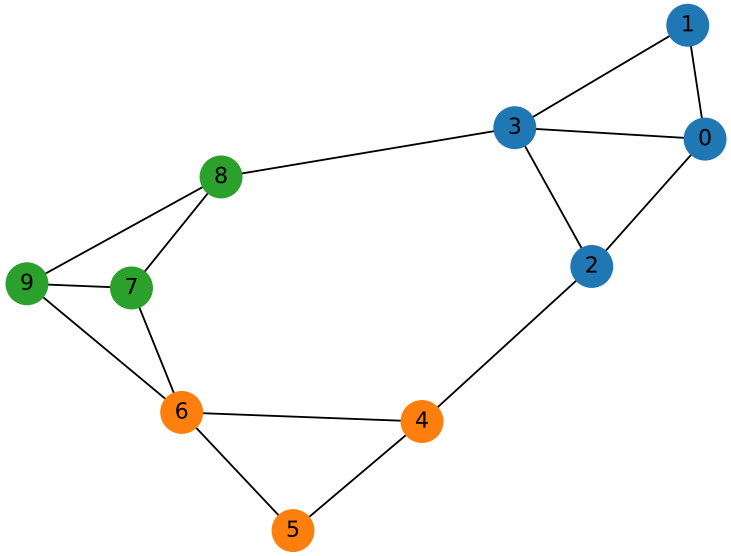

Figure 8: Optimal found solution from H-QAOA on `HomeMadeGraph1` instance. Q=0.39 with k=3 clusters.

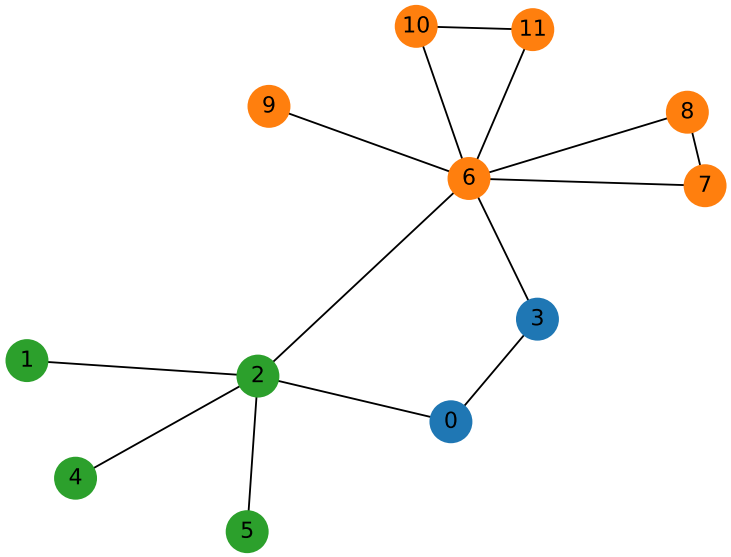

Figure 9: Optimal found solution from H-QAOA on `HomeMadeGraph2` instance. Q=0.36 with k=3 clusters.

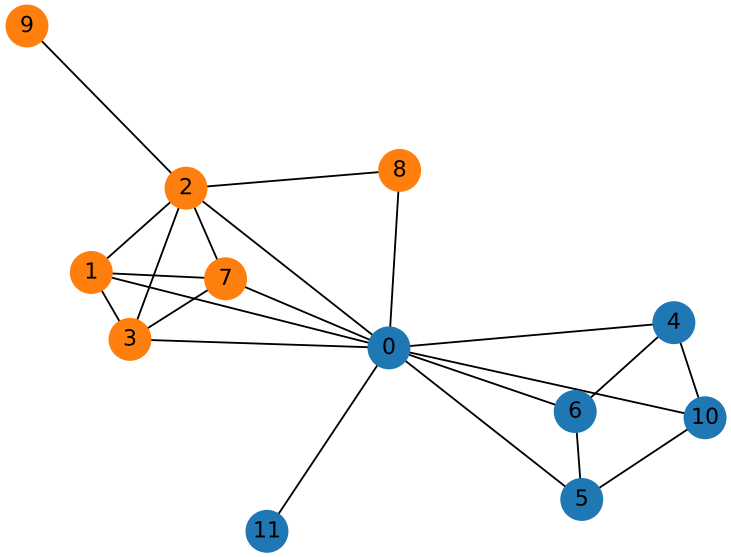

Figure 10: Optimal found solution from H-QAOA on `SmallKarateClub` graph instance. Q=0.27 with k=2 clusters.

B Heatmaps

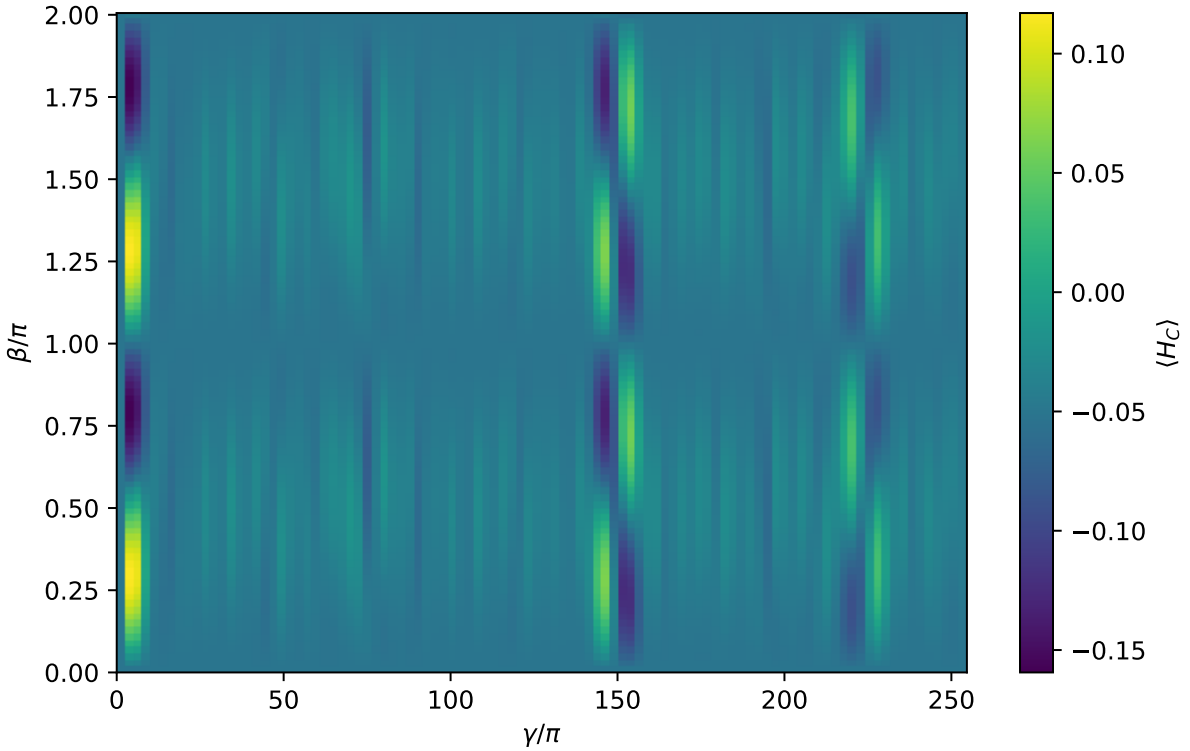

Figure 11: Heatmap of $\langle H_C|H_C\rangle$ given γ and β values for the HomeMadeGraph1 instance.

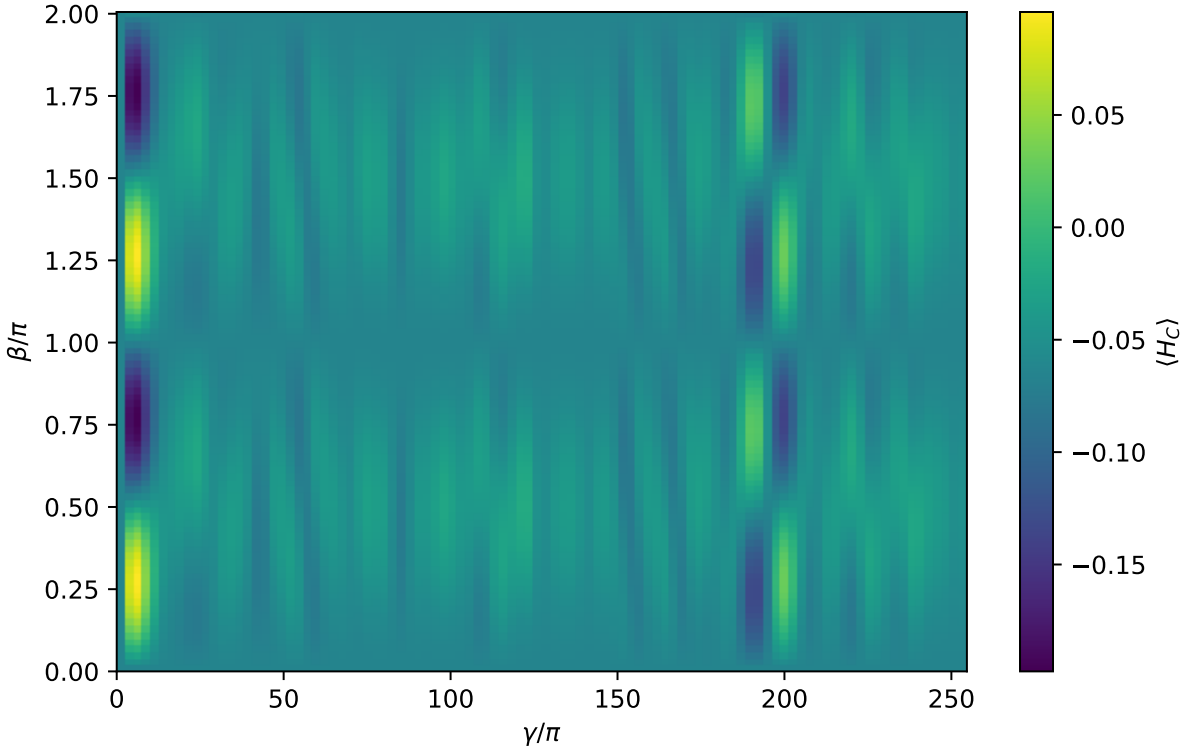

Figure 12: Heatmap of $\langle H_C|H_C\rangle$ given γ and β values for the HomeMadeGraph2 instance.

C Declaration of AI and LLM Usage

For this project, Large Language Models (LLMs) were utilized for code debugging, logic refinement, function creation, structural organization, text editing, literature searching, and general brainstorming. All LLM-assisted code functions have been documented in the project's GitHub repository.

Regarding the manuscript itself, Table 2 outlines the level of AI contribution for each section. The formal definitions for these contribution levels (CL) can be found in this repository: https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/LLM_Usage_Declaration_Guidelines.md.

Section	LLM CL	Notes
Abstract	3	Drafted by LLM based on full text, revised by me.
Introduction	2	Rewrote three specific paragraphs.
Methods	2	Restructured sections for clarity.
Code and Benchmarking	1	Minor phrasing corrections.
Results and Discussion	2	Improved clarity and flow of arguments.
Conclusions and Future Work	1	Grammar and minor phrasing.

Table 2: LLM CL denotes the Large Language Model Contribution Level.

Note on Abstract (Level 3): Claude was provided with the full text of the manuscript and prompted to generate an initial abstract. The final version was polished by the me to refine sentence structure and eliminate redundancies.

Tools Utilized

- Claude (Sonnet 4.6, accessed via https://claude.ai/, April-May-June 2026)
- Gemini (3.5 Flash, accessed via https://gemini.google.com/, April-May-June 2026)
- GitHub Copilot (integrated in VS Code, throughout the project)